# Software Project Management

# Introduction

# What is a project?

What's a project?

- PMI (Project Management Institute) definition
  - *A project is a temporary endeavor undertaken to create a unique product or service*
- Description: A sequence of connected and related activities ***(requirement engineering, system engineering, coding, testing, documentation, controlling, …)*** that must be completed by a specific time, within budget, and according to specification.
- Progressively elaborated
  - With repetitive elements
- A project manager
  - Analogy: conductor, coach, captain

# Project vs. Program Management

What's a 'program'?

- Mostly differences of scale
- Often a number of related projects
- Longer than projects
- Definitions vary
- Example: Program Manager for MS Office
  - Includes projects: Spelling, drawing, Word, Excel, PowerPoint, etc.

# Project characteristics

- A project is unique
  - A Project creates unique deliverables: product, service, or result
  - Even technically-identical projects are distinctly unique, due to internal or external contingent factors
- Projects are temporary
  - Every project has a definite beginning and a definite end
  - Project end may be reached through success, qualified success, failure, or redundancy
  - Projects need not be of short duration, but they are of finite duration

# Project characteristics

- Projects have customer-specified performance criteria
  - The customer defines these criteria in the form of requirements
  - We will see that **requirements** are one of the **most critical characteristics** of a project that contributes to its **success or failure**
- Projects consume resources
  - Personnel resources
  - Physical resources
  - Monetary resources

# Projects *vs. operations/production*

**Similarities**

***Both Projects & Operations***

- Consume resources
- Are constrained by budget, time, resources, etc.
- Are planned, executed, & controlled

***Projects***

- Are unique
- Are temporary
- *Example*: Developing v1.0 of a new cloud-based software product (*e.g.*, Adobe Creative Cloud)

**Differences**

***Operations***

- Are repetitive
- Are on-going
- *Example*: Maintenance and delivery of cloud-based software product

# Characteristics of IT (Information Technology)

- IT encompasses all forms of technology used to create, store, exchange, and use information in various forms
- ‘Information’ includes conventional data, voice, images, multimedia, etc.
- Of central importance are computer, software, and communications technologies
- Virtually all significant projects are now distributed: networks of computers communicate only via message passing
- Distribution poses additional challenges in IT projects: reliability, availability, security, and information synchronization

# How do IT projects differ from ‘ordinary’ projects?

- IT products and services possess greater complexity
  - IT products and services are basically complex by nature
  - Computing + communication + diverse data ⇒ complexity
- IT projects have tight schedules
  - Tight schedules are the norm in IT development
  - Scheduling is aggravated by a pervasive ‘rush to market’ mentality
- IT is an integral part of enterprise infrastructure
  - IT is no longer an auxiliary element of the enterprise
  - A business’ success is often critically dependent upon IT support
- IT is permeated by quickly-changing technology
  - Technology evolves (and may even become obsolete) during the lifetime of a major project

# Project characteristics

- Four characteristics of projects:
  - finite time
  - people assigned
  - clear roles and responsibilities
  - things to deliver
- Have you ever had this feeling about a project?
  - not enough time
  - too few people
  - people not sure what they should be doing
  - too much to do

# How Are Software Projects Different?

- Consists of hardware and software.
- Software is [usually] custom written and one-of-a-kind.
- Hard to determine progress. [One can see how far the Freedom Tower (World Trade Center) has progressed. One cannot see how far a piece of software has progressed]
- Difficult to estimate schedule.
- Difficult to determine cost.

# What is Project Management?

- **Project management** is “the application of knowledge, skills, tools and techniques to project activities to meet project requirements”
- Software Project Management is the art to define, plan, execute, and monitor the activities that will bring **software products** to existence.
- Project managers strive to meet the **triple constraint** by balancing project scope, time, and cost goals

# Project management processes

- Regardless of the type of project lifecycle, project management encompasses the following process groups, shown with some representative tasks:
  1. **Initiating/Define** – Scope the project; Charter the project; identify stakeholders
  2. **Planning** – Develop the project plan**.** Collect requirements; identify schedule; plan scope, cost, quality, human resource, risk, and procurement management
  3. **Executing** – Launch the plan**.** Direct and manage project work; perform quality assurance; manage and develop project team; conduct procurements
  4. **Monitoring and Controlling** – Monitor project progress. Monitor and control project work; manage scope change; monitor and control schedule; control quality; control risks; control procurements
  5. **Closing** – Close out the project: Close project; close procurements

# Software Project

**Quality Engineering Principle:**

- The quality of the software system is controlled by the quality of the process used to produce that software.

**Quality Management Principle:**

- Document the process
- Measure the process
- Improve the process based on the measurement

# Software Development Process

- What is the software development process?
  - A process is a **set of documented** procedures, methods, practices, and tools used to produce a software product.
- The process will answer the following:
  - What to do? **Tasks/activities**
  - How to do it? **Procedure/practice**
  - When to do it? **Sequence of activities**
  - What are the artifacts? **(input/output)**

# Software Project

- If the programmer and designer follow the **process,** then the artifacts they produce will be
  - Predictable
  - Based on the requirements
  - Easy to maintain and control
  - Consistent with the writing style
  - Of acceptable quality
  - Within acceptable milestones
- By following the process, we will be able to know precisely what/how/when/where it happened !

# Software Processes

Software Process is an overloaded term

- **Metaprocess:** an organization's policies, procedures, and practices for pursuing a software-intensive line of business; the focus is on organizational economics, and long-term strategies.
- **Macroprocess:** the project's policies, procedures, and practices for producing a complete software product within certain cost, schedule, and quality constraints.
- **Microprocess:** a project team's policies, procedures, and practices for achieving an artifact of the software process.

# Core project management activities

- Identify requirements
- Address the various needs, concerns, and expectations of the stakeholders in the project
- Establish active, effective, collaborative communications among the stakeholders
- Manage stakeholders towards meeting project requirements and creating project deliverables
- Balance the competing project constraints, including: scope, quality, schedule, budget, resources, and risks

# Software Project Trade-offs

**What is the goal?**

- Balance the main three in order to:
  - Stay within the budget (cost)
  - Deliver on time to gain market share (time)
  - Exceed customer satisfaction (quality)

**The time/scheduling hypocrisy**

- What can you tell me about the next project, you ask?
  - **It is due on June 1st tells your manager**
- We hold deadlines too dearly. Of course, time to market is critical
- But what generally happens on projects when you hit that deadline?

# What makes a project successful?

Successful project management means meeting all three goals (scope, time, cost) – **and** *satisfying the project's sponsor*.

**Trade-off Triangle**

- **Project constraints:** Fast, cheap, good. Choose two. Also stated as: "On-time, on-budget, high-quality. Choose two."
  - **Reality often ignored in project planning**
- Know which of these are fixed & variable for every project
  - Time
  - Cost
  - Scope



# Success Metrics

1. On schedule
   - Requires good: plan; estimation; control
2. Within budget
   - Again: planning, estimation & control
3. According to requirements
   - Importance of good requirements
   - Perception & negotiation critical
4. High quality. May or may not be same as item 3

**Only real measure:**

**Is the customer happy?**

Customer satisfaction!!

# Factors in Project Success & Failure

# Software Crisis

- Many software-related failures: auto-pilot systems, air traffic control systems, banking systems, IRS (Interface Requirements Systems).
  - On January 15, 1990, the AT&T long-distance telephone network broke down, interrupting long-distance telephone services in US for over 8 hours. [Missing **`break`** in a **`switch`** statement.]
  - On June 4, 1996, the maiden flight of the new and improved Ariane 5 rocket exploded 37 seconds after lift-off.
  - On June 8, 2001, a software problem caused the NYSE to shut down the entire trading floor for over an hour.
  - Many, many, many more.

# What is the problem?

**Software Projects have a terrible track record**

A 1995 Standish Group study (CHAOS) found that only 16.2% of IT projects were successful in meeting scope, time, and cost goals (on-time & on-budget) [Things have improved a bit since.]

Over 31% of IT projects were canceled [never seeing completion], costing over $81 billion in the U.S. alone

- They never worked
- Too late for the market window

Most projects are

- Late in delivery
- Missing functionality
- Have major defects (bugs)
- Did not do what the customer wanted
- Hard to maintain and support

# The Standish Group's CHAOS Report (2001)

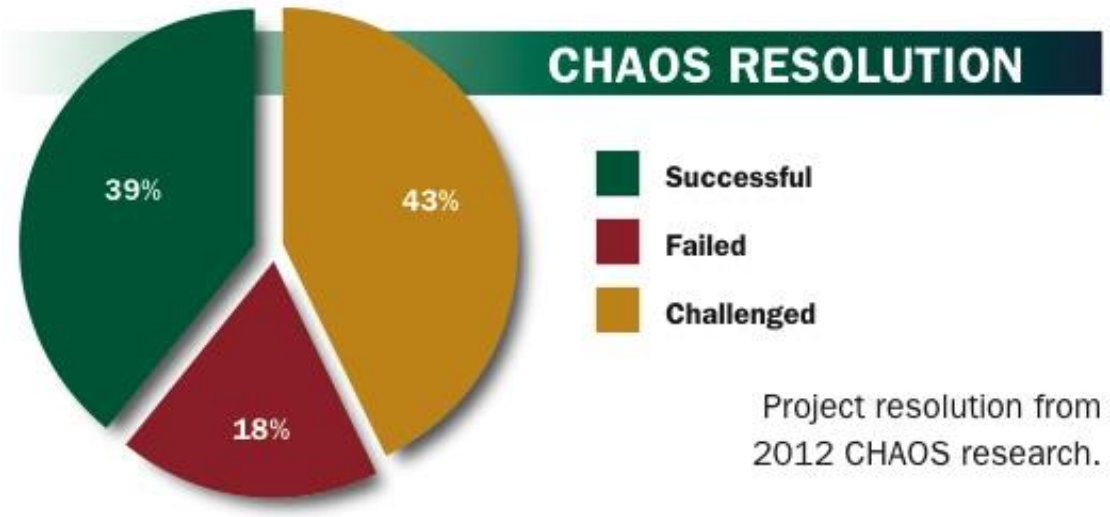
- Standish compiles and publishes a periodic survey on the success and failure rates of IT projects
- These statistics date from **2001**–however, based on more recent complementary reports, these numbers have remained fairly stable over the years:
  - Average IT project schedule overrun: 163%
  - Average IT project cost overrun: 145%
  - Actual deliverable functionality compared to plan: 67%
  - IT projects judged a success: 26%
  - Lost value from marginal and failed projects: $75 billion

# Chaos Report 2012

**Project Success:** Type 1. The project is completed on-time and on-budget, with all features and functions as initially specified. **(2012: 39%)**

**Project Challenged:** Type 2. The project is completed and operational but over-budget, over the time estimate, and offers fewer features and functions than originally specified. **(2012: 43%)**

**Project Impaired:**
Type 3.
The project is canceled at some point during the development cycle. **(2012: 18%)** (Are ALL impaired projects failures???)



# What Went Right? – Improved Project Performance

- The Standish Group's CHAOS studies show improvements in IT projects in the past decade

**RESOLUTION**

| | 2004 | 2006 | 2008 | 2010 | 2012 |
|---|---|---|---|---|---|
| **Successful** | 29% | 35% | 32% | 37% | 39% |
| **Failed** | 18% | 19% | 24% | 21% | 18% |
| **Challenged** | 53% | 46% | 44% | 42% | 43% |

Project resolution results from CHAOS research for years 2004 to 2012.

# Why the Improvements?

"The reasons for the increase in successful projects vary. First, the average cost of a project has been more than cut in half. Better tools have been created to monitor and control progress and **better skilled project managers with better management processes** are being used. The fact that there are processes is significant in itself.” *

*The Standish Group, "CHAOS 2001: A Recipe for Success" (2001).

# A Case Study
# The Initial Launch of HealthCare.gov (2013)



# ACA – HealthCare.gov

- ACA (**Affordable Care Act** ) signed into law on March 23, 2010
- **HealthCare.gov** is a healthcare exchange website.
  - "One-stop shopping sites for health insurance"
  - CBO (Congressional Budget Office) forecast: 7 million users during the first year
- Development contracts awarded in September 2011
  - No-bid, cost-plus contracts
  - Pre-certified private contractors
- HealthCare.gov launched on October 1, 2013
  - Serious technological problems

# HealthCare.gov – The Launch Problems

- Performance: response time (landing page) > 8s
  - “Annoyingly long wait times"
- Navigation: broken UI
- Stability: intermittent crashes, availability ≈ 43%
- Functionality: incorrect and incomplete data
- Error rate (per page) ≈ 6%
- Scalability: < 1,100 concurrent users
- Enrollment completion rate < 30%

# HealthCare.gov – The Contractors & The Cost

- The lead contractor: CGI Group
  - At least 47 private companies involved
  - Including QSSI, Equifax, Serco
- Coordinated by the Centers for Medicare and Medicaid Services (CMS)
- Total budget: $293 million
  - CGI: $196 million (2013). $112 million paid Oct. 2013
  - QSSI: $85 million
- Estimated actual cost: > $500 million by Oct. 2013

# HealthCare.gov – The Failures – Software Eng.

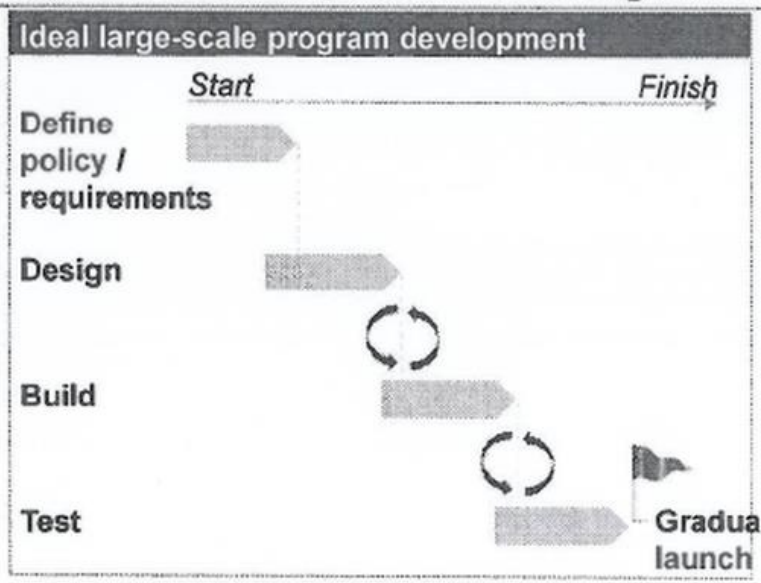
- **Inadequate Testing**
  - “This system just wasn't tested enough.” – CMS
  - Full test began T -2 weeks (time before launch).
  - Final “pre-flight checklist” T -1 week: 41 of 91 functions fail.
  - No “end-to-end” test as late as T -4 days
  - Stress tests T -1 day: performance degradation with only 1,100 concurrent users. (50,000-60,000 expected)
  - Final top-to-bottom security tests not finished.
  - No integration test. No beta test.

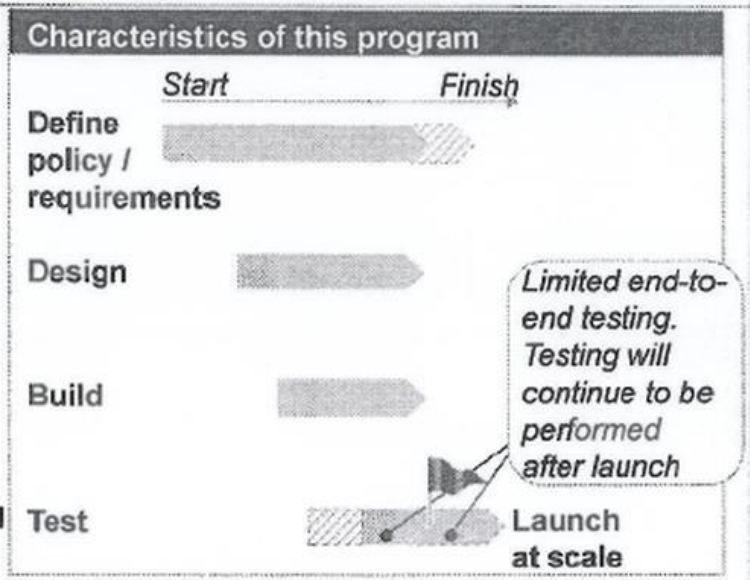
# HealthCare.gov – The Failures – Software Eng.

- **Evolving, Rolling Requirements**
  - Regulations and policies were still in flux when contracts awarded in 2011.
  - The specifications for the project were delayed repeatedly.
  - The regulations and policies were modified repeatedly until summer 2013.
  - Repeated changes result in design changes.
  - CGI (Common Gateway Interface) did not start coding until Spring 2013
- **Failure to Effectively Manage Changes**
  - "Write-down-all-the-requirements-then-build-to-those-requirements"
  - Did not adopt an agile development approach.
  - Committed to an all-or-nothing launch date.

# Case Study: HealthCare.gov– McKinsey "Red Team" Assessment

## Programs of this type ideally have a sequential planning, design, and implementation process with significant testing and revision




**Description of ideal situation:**

- Clear articulation of requirements & success metrics
- Minimized dependency on third parties
- Sequential requirements, design, build, and testing
- Iteration and revision between phases
- End-to-end integrated operations and IT testing
- Limited initial launch

**Current situation:**

- Evolving requirements
- Multiple definitions of success
- Significant dependency on external parties/ contractors
- Parallel "stacking" of all phases
- Insufficient time and scope of end-to-end testing
- Launch at full volume

**CMS has been working to mitigate challenges resulting from program characteristics**

CMS

*Confidential and Proprietary — Pre-decisional Information*

# Case Study: HealthCare.gov: The Failures – Management

- Management expertise is in getting contracts not delivering projects.
- Project quality is sacrificed for the sake of appearances.
- Seriously substandard staffing and under staffing
  - CGI. Three months before launch, only 10 developers were working on a crucial part of the site, and of those, only one was "at a high enough skill level."
- Lack of coordination among contractors
  - Unclear responsibilities. Fragmented authority.

# Case Study: HealthCare.gov: The Failures – Gov. & Policies

- Government IT projects
  - Most are over budget and/or behind schedule.
  - “Write-down-all-the-requirements-then-build-to-those-requirements” is outdated.
- IT procurement policies
  - Cost-plus contract, no-bid contracts
  - “The firms that typically get contracts are the firms that are good at getting contracts, not typically good at executing on them.”

# Case Study: HealthCare.gov: The Lessons Learned

- Adopt software engineering best practices
  - Agile software development. Testing early.
  - Software quality assurance and testing. Testing throughout.
- Adopt management best practices
  - Clear responsibility and accountability
  - Performance metrics and progress tracking
- Revamp government IT procurement policies
  - Current system is antiquated, and has failed.
  - Bring government IT to the 21th century.

# Case Study: HealthCare.Gov – Dec. 2013

- 400+ bug fixes, by the end of Nov. 2013
  - "Operate smoothly for most users." – W.H.
- Availability > 90%
- Response time (landing page) < 1s
- Error rate (per page) < 1%
- Completion rate ≈ 80%
- System capacity ≈ 50,000 concurrent users
- Sign-ups
  - 27,000 in Oct, 110,000 in Nov, 975,000 in Dec

# Case Study: HealthCare.Gov – April 2014

- Issues on data accuracy and completeness
  - Estimated 10-15% of sign-ups missing: outbound 834
  - Other inaccuracies have been reported
- CGI work during repair continue to be substandard
  - Half of the software fixes failed. – *CMS*
- CGI contract has been terminated. (Jan. 10 , 2014)
- March 31, 2014 (last day to sign up), site went down
- April 1, 2014. 7.1 million signed up – *W.H.*

# Case Study: HealthCare.Gov – Aftermath

- *Accenture* took over in Jan. 2014 as the lead contractor for development and maintenance: $175M
- Cost of building the system – *GAO,* July 2014
- $834M through Feb. 2014
  - Total estimated cost: > $2B
- "CMS undertook the development of HealthCare.gov without effective planning or oversight practices"
- Also found "increased and unnecessary risk of unauthorized access, use, disclosure, modification or loss" of information
- CMS only withheld $267,000 in requested fees, 2% of the contract, from CGI.

# Case Study: HealthCare.Gov – 2015 Enrollment Cycle

- Enrollment for 2015 (37 states)
  - Nov 15, 2014 – February 15, 2015.
  - Outages on the first day
  - More smooth operation thereafter
- February, 2015
  - ~ 11.4 million sign-ups (~8.6 million re-enrollments)
    - Open enrollment extensions: March 15 – April 30
  - ~800,000 received incorrect tax information,
    - Incorrect amount on 1095-A Form

# What is the problem?

**Ever-Present Difficulties**

- Few guiding scientific principles
- Few universally applicable methods
- As much people problems as technological
  - managerial / psychological / sociological
- Sponsors unwilling to spend money for supposedly unrewarding activities
  - Quality
- Organizational rivalries
- Time pressure
- Cost pressure

# Why do IT projects fail?

- Requirements, requirements, requirements
  - Requirements are unclear, incomplete, or the project management methodology does not accommodate changing requirements effectively
- Lack of user involvement
  - A disengaged or absentee user (~customer) is an invitation for project problems or outright failure
  - Often the result of a lack of business and IT integration
- Unrealistic expectations (including unrealistic time frame expectations)
  - Closely related to/result of ‘Lack of user involvement’
  - “How hard can it be?” attitude of business and technical management
  - Overly optimistic ‘can do’ attitude at all levels
  - It is the project management team's responsibility to educate users in the realities of the project

# Why do IT projects fail?

- Lack of planning
  - You cannot achieve any goal without planning
  - Proper approach to planning in a complex project must be adaptive: big planning up front is a waste of time and gives a false sense of security
  - Related issues: Perceived rush to get started; overconfidence
- Unclear (or lack of) vision and/or objectives
  - The project vision must be established as one of the first steps in project planning
  - The vision for a project should be short, concise, and laser-sharp: the ‘elevator statement’ format is most effective

# Why do IT projects fail?

36 Classic Mistakes

- AntiPatterns
  - Seductive Appeal: good reason for decisions at the time
- Types
  - People-Related
  - Process-Related
  - Product-Related
  - Technology-Related
- Gilligan's Island

# People-Related Mistakes Part 1

- Undermined motivation
- Weak personnel
  - Weak vs. Junior
- Uncontrolled problem employees
- Heroics
- Adding people to a late project
- Lack of match between people and needs
  - Incompetent or over competent
  - Culture clash

# People-Related Mistakes Part 2

- Noisy, crowded offices
- Customer-Developer friction
  - Unrealistic expectations
  - Lack of user input
- Politics over substance
- Lack of effective project sponsorship
- Lack of stakeholder buy-in
- Wishful thinking

# Process-Related Mistakes Part 1

- Optimistic schedules
  - Omitting necessary tasks from estimates
  - Planning to catch-up later
  - Code-like-hell programming
  - Insufficient risk management
- Contractor failure
- Insufficient planning
- Abandonment of plan under pressure

# Process-Related Mistakes Part 2

- Wasted time during fuzzy front end
- Shortchanged upstream activities
- Inadequate design
- Shortchanged quality assurance
- Insufficient management controls
- Frequent convergence

# Product-Related Mistakes

- Requirements gold-plating
  - Gilding the lily
- Feature creep
- Developer gold-plating
  - Beware the pet project
- Push-me, pull-me negotiation
  - Slip schedule + add features
- Research-oriented development

# Technology-Related Mistakes

- Silver-bullet syndrome
- Overestimated savings from new tools and methods
  - Fad warning
- Switching tools in mid-project
- Lack of automated source-code control

# What Helps Projects Succeed?*

1. Executive support
2. User involvement
3. Experienced project manager
4. Clear business objectives
5. Minimized scope
6. Standard software infrastructure
7. Firm basic requirements
8. Formal methodology
9. Reliable estimates
10. Other criteria, such as small milestones, proper planning, competent staff, and ownership

*The Standish Group, “Extreme CHAOS,” (2001).

# Four Project Dimensions

**Four Project Dimensions (The 4 P's)**

- **People** — the most important element of a successful project
- **Product** — the software to be built
- **Process** — the set of framework activities and software engineering tasks to get the job done
- **Project** — all work required to make the product a reality

# People

"It's always a people problem"

– Gerald Weinberg, "The Secrets of Consulting"

- Developer productivity: 10-to-1 range
  - Teams 3 (or 5) to 1 difference
- Improvements:
  - Team selection
  - Team organization
  - Motivation
- Other success factors
  - Matching people to tasks
  - Career development
  - Balance: individual and team
  - Clear communication

# Process

- Is process stifling?
  - 2 Types: Management & Technical
- Development fundamentals
- Quality assurance
- Risk management
- Lifecycle planning
- Avoid abuse by neglect
- Customer orientation
- Process maturity improvement
- Rework avoidance
- **Goals**
  - cut time-to-market
  - Improve quality

# Product

- The “tangible” dimension
- Product size management
- Product characteristics and requirements
- Feature creep management

# Technology

- Often the least important dimension
- Language and tool selection
- Value and cost of reuse

# Planning

- Determine requirements
- Determine resources
- Select lifecycle model
- Determine product features strategy
- **Tracking**
  - Cost, effort, schedule
  - Planned vs. Actual
  - How to handle when things go off plan?

# Measurements

- To date and projected
  - Cost
  - Schedule
  - Effort
  - Product features
- Alternatives
  - Earned value analysis
  - Defect rates
  - Productivity (ex: SLOC)
  - Complexity (ex: function points)

# Technical Fundamentals

Assumed Software Development Life Cycle (SDLC)

- Requirements
- Analysis
- Design
- Construction
- Quality Assurance (aka Testing)
- Deployment

# Software Project Management Fundamentals

# Formal Project Management

**Advantages of Using Formal Project Management**

- Better control of financial, physical, and human resources
- Improved customer relations
- Shorter development times
- Lower costs
- Higher quality and increased reliability
- Higher profit margins
- Improved productivity
- Better internal coordination
- Higher worker morale (less stress)
  - Less “death marches”
  - Less overworked personnel

# Getting organized

So, … now what?

- Who is involved?
  - Stakeholders
- What do they want done?
  - Charter, vision, requirements
- Who do we have available to do the work?
  - Resources and staffing
- How do we do this?
  - Project planning, WBS
- How much will it cost
  - Estimating
- When will it be finished?
  - Scheduling
- What can possibly go wrong?
  - Risk Management

# Summary of essential points

- Projects and operations have both similarities and differences
- Complex projects exhibit highly contingent behavior, are unpredictable, and face complex risks
- Virtually all IT projects should be considered complex projects
- Complex project management requires integration of significant sociological, psychological, and technical skills
- The factors contributing to project success and failure are reasonably well-known
- This course discusses ways to achieve project success factors